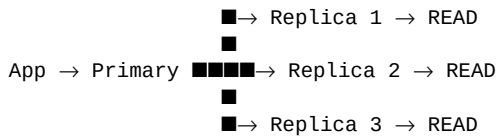


# Read Replicas & Database Replication

Hinglish • Practical System Design • Production + Interview Notes

## 1. Read Replica Kya Hai?

Read replica primary database ki replicated copy hoti hai. Primary par writes hoti hain aur replicas ko commonly read traffic handle karne ke liye use kiya jata hai.



WRITE → Primary

## 2. Read Replicas Ki Zarurat Kyun?

Agar workload heavily read-oriented hai, single database par reads primary ka CPU, memory, I/O aur connection capacity consume kar sakti hain. Replicas read workload ko distribute karne mein help karti hain.

Traffic:

1000 req/sec

■ 900 READ

■ 100 WRITE

Without replicas:

900 READ + 100 WRITE → Primary

With replicas:

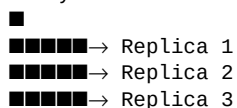
100 WRITE → Primary

900 READ → Replica pool

## 3. Replication Kya Hai?

Replication ka matlab database state/changes ko ek database instance se doosre instances tak copy karna hai. Exact mechanism database engine par depend karta hai.

Primary



Replication read scaling ke saath availability/failover architecture mein bhi role play kar sakti hai, but replication alone backup ka replacement nahi hai.

## 4. Primary vs Replica

| Role         | Typical responsibility                                           |
|--------------|------------------------------------------------------------------|
| Primary      | Writes + reads that require strong/read-after-write behavior     |
| Read Replica | Read workload; may lag behind primary                            |
| Backup       | Independent recovery copy / restore mechanism; different purpose |

## 5. Replication Lag

Primary par write hone aur replica par woh write visible hone ke beech ka delay replication lag hai.

```
User
↓
WRITE █500
↓
Primary DB
↓
replication
↓
Replica
```

If read happens before replica catches up:  
→ old value possible

Lag milliseconds, seconds, ya workload/architecture ke according more ho sakta hai. Exact behavior database and replication setup par depend karta hai.

## 6. Read-After-Write Problem

User ne data update kiya aur immediately same data read kiya. Agar write primary par aur read lagging replica par gaya, user ko old data mil sakta hai.

```
POST /profile
↓
Primary → name updated

GET /profile
↓
Replica → old name ← possible
```

Common solutions: critical read ko primary par route karna, session/request ko temporarily primary-read mode mein rakhna, ya architecture ke consistency guarantees use karna.

## 7. Read/Write Splitting

```
Node.js / DB proxy
█
███ WRITE ███→ Primary
█
███ READ ███→ Replica pool
```

Application, database proxy, ya managed database routing layer read/write split kar sakti hai. Important: har GET ko blindly replica par bhejna safe nahi hota.

## 8. Kaunse Reads Replica Par?

Usually stale data tolerate karne wale reads replicas ke good candidates hote hain: product catalog, public content, analytics-style dashboards, historical listings, etc. Exact choice business consistency requirement par depend karti hai.

Payment status, wallet balance, inventory immediately after a critical write, or other strict read-after-write paths ko special handling ki zarurat ho sakti hai.

## 9. Replicas Sirf Reads Ke Liye?

Read replicas ka primary purpose read scaling hai. Kuch database technologies/configurations replicas ko other workloads ke liye bhi support kar sakti hain, but architecture ko product-specific capabilities ke according design karna chahiye.

Interview mein simple answer: 'Read replicas primarily read traffic scale karne ke liye use karunga; writes primary par rahengi unless database architecture explicitly supports another write topology.'

## 10. Replication Lag Monitor Kaise Karoge?

Production mein lag ko monitor karna important hai. Metrics database-specific hoti hain, but conceptually primary ke latest position aur replica ke applied/replayed position ke gap ko monitor kiya jata hai.

```
Monitoring
↓
Replication lag
↓
healthy? ■■→ route reads normally
■
high lag
↓
reduce/stop replica reads
or route sensitive reads to primary
```

## 11. Replica Failure

Agar ek replica fail ho jaye, load balancer/proxy/router ko healthy replicas par traffic continue karna chahiye. A single replica failure ideally entire application ko down nahi karta.

```
Replica 1 ■
Replica 2 ■
Replica 3 ■

Read router
↓
2 + 3 handle traffic
```

## 12. Primary Failure & Failover

High-availability setup mein primary failure par eligible replica ko new primary promote kiya ja sakta hai. Automated failover exact database/managed service setup par depend karta hai.

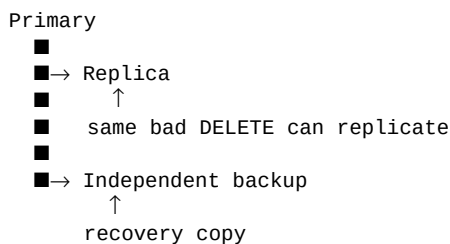
```
Normal:
App → Primary
      ■→ Replica 1
      ■→ Replica 2

Primary fails
↓
Failover
↓
Replica 1 → New Primary
```

Failover ke baad application ko correct endpoint/discovery, retry behavior, transaction safety aur connection recovery handle karni hoti hai.

## 13. Replication vs Backup

Ye interview ka common trap hai: replication backup nahi hai. Agar primary par accidental DELETE hua aur woh change replicate ho gaya, replica bhi affected ho sakta hai.



Production mein replication + independent backups + restore testing alag responsibilities cover karte hain.

## 14. Synchronous vs Asynchronous Replication

Asynchronous replication mein primary write acknowledge kar sakta hai before replica has fully applied the change, so lag possible hai. Synchronous approaches stronger consistency provide kar sakti hain but write latency/availability trade-offs la sakti hain.

Async:  
Primary write → ACK  
→ replica catches up later

Sync-style:  
Primary write  
↓  
required replica acknowledgement  
↓  
ACK to client

Exact semantics database technology par depend karti hain. Interview mein consistency vs latency trade-off mention karna important hai.

## 15. Replicas Aur Connection Pool

Replicas add karne ka matlab unlimited connections create karna nahi hai. Node instances × pool size × replicas ko consider karna padta hai.

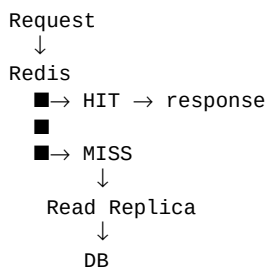
10 Node servers  
× 50 DB connections  
= up to 500 connections

If distributed across DB endpoints,  
total connection pressure still matters.

DB connection limits, pool size, traffic pattern and query concurrency ko together tune karo.

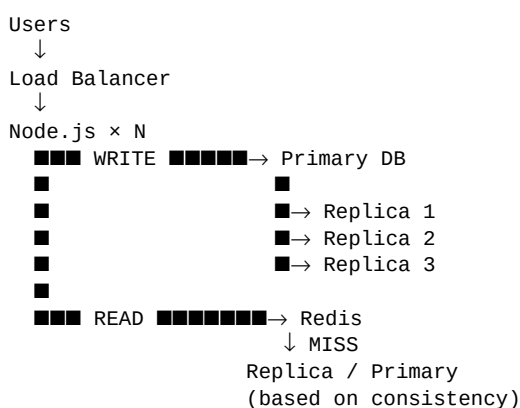
## 16. Read Replica + Redis

Redis aur read replicas competitors nahi hain; different layers par kaam karte hain. Redis repeated hot reads ko database tak pahunchne se pehle serve kar sakta hai. Replica remaining database reads ko scale kar sakti hai.



Typical cache miss path primary ya replica dono ho sakta hai depending on consistency requirement.

## 17. Production Architecture



## 18. When Read Replicas Are NOT Enough

Agar workload write-heavy hai, read replicas primary write bottleneck solve nahi karenge. Query optimization, indexes, batching, caching where applicable, vertical scaling, partitioning/sharding, queueing, or data-model changes may be needed.

Problem:  
80% WRITE / 20% READ

Adding 5 read replicas  
→ doesn't remove primary write bottleneck

## 19. Common Mistakes

1. Har read ko replica par bhejna.
2. Replication lag ignore karna.
3. Replica ko backup samajhna.
4. Read replicas ko write scaling solution samajhna.
5. Unlimited DB connections create karna.
6. Failover ke baad application connection recovery test na karna.
7. Replica lag/high error rate ko monitor na karna.

## 20. Interview Q&A;

| Question                              | Short answer                                                                                      |
|---------------------------------------|---------------------------------------------------------------------------------------------------|
| Read replica kya hai?                 | Primary database ki replicated copy, mainly read workload scale/availability ke liye.             |
| Why read replicas?                    | High read traffic ko multiple database nodes par distribute karne ke liye.                        |
| Write kahan?                          | Typically primary par.                                                                            |
| Replication lag?                      | Primary change aur replica visibility ke beech delay.                                             |
| Old data kyun dikha?                  | Read lagging replica par route hua.                                                               |
| Read-after-write kaise handle?        | Sensitive read ko primary par route or suitable consistency strategy use.                         |
| Replica fail?                         | Healthy replicas par traffic shift; monitor and replace/recover failed replica.                   |
| Primary fail?                         | HA/failover mechanism eligible replica ko promote kar sakta hai.                                  |
| Replication = backup?                 | No. Independent backups are still needed.                                                         |
| Sync vs async?                        | Sync stronger acknowledgement/consistency behavior but may add latency; async allows lag.         |
| Read replicas write bottleneck solve? | No, primarily read scaling.                                                                       |
| Redis + replicas?                     | Redis repeated hot reads reduce karta hai; replicas remaining DB read load distribute karti hain. |

## 21. Practice Scenarios

1. 90% reads, 10% writes. Design the database layer.
2. User updates profile and immediately sees old profile. Explain why.
3. Replica lag suddenly increases from milliseconds to seconds. What would you check?
4. One replica goes down. How should the application behave?
5. Primary database fails. Describe a high-level failover flow.
6. Product listing can tolerate slightly stale data, but payment status cannot. Where would you route each read?
7. Why doesn't adding read replicas solve a write-heavy workload?
8. Why are replicas not a substitute for backups?
9. How would Redis and read replicas work together?
10. What metrics would you put on a database replication dashboard?

## 22. Final Cheat Sheet

### READ REPLICA CHEAT SHEET

#### Primary:

- writes
- critical/read-after-write reads when needed

#### Replica:

- read scaling
- may lag

#### Replication lag:

- primary changed
- replica catches up later

#### Watch out:

- stale reads
- replica overload
- connection limits
- failover
- backup ≠ replication

Mental model:

WRITE → Primary

READ → Redis / Replica

↓

Primary when strong/read-after-write  
consistency is required

Golden interview line:

"Read replicas reduce primary read load,  
but I will account for replication lag and  
route consistency-sensitive reads appropriately."

## 23. Local Practice

Local environment mein primary + replica behavior simulate karo ya database-supported replication lab use karo. Phir Node.js mein read/write routing implement karo, artificial delay se replication lag simulate karo, aur observe karo ki immediate reads stale data kaise de sakti hain. Redis cache ko add karke architecture compare karo.